

1. * Wrong sign flag causes incorrect positive/negative result interpretation
* Wrong zero flag may falsely indicate zero output
* proper flag updating and signed arithmetic checking ensure correct subtraction results
-

2. * Memory-based operands requires extra access cycles, causing slower execution and update delays.
* Using register operands improves speed and maintains accurate arithmetic calculation
-

3. * Repeated NOT operations continuously flip bits, causing loss of the original signal state
* Without reference values, the controller may misinterpret signals and make wrong decisions.
* store the original signal or use a status/reference register to maintain correct control logic.
-

4. * Incorrect AND masking can remove valid signal bits along with noise, causing data loss and wrong output.
* choosing the correct mask value preserves required bits while filtering unwanted noise accurately.
-

5. *Ignoring carry and overflow causes cumulative arithmetic errors.
*Incorrect values propagate through continuous operation.
*Regular flag checking and error validation ensures accurate calculations.

6. *Incorrect handling of arithmetic instructions and flags causes overflow and wrong results
*Large or memory based operands reduce execution accuracy and increase errors.

7. *XOR operation is reversible; so applying the same key restores the original data
*Using different keys during encoding and decoding produces incorrect outputs.
*Store and apply a consistent key for both operation to ensure accurate data transformation.

8. *Improper OR operation can activate outputs unintentionally, when any overlapping input becomes HIGH
*Correct logical conditions and proper input validation prevent unwanted control outputs.
